Programa:	PDF Password Remover v2.2
Protección:	UPX + Límite de 100 usos
Objetivos:	Desempakar y evitar la limitación
URL descarga:	http://www.verypdf.com/pwdremover
Dificultad:	Fácil
Herramientas:	OllyDbg 1.10 + PEiD 0.93 + UEdit32 + Import Reconstructor 1.6
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 21-10-2005

Hola a todos..!!

Una vez más volvemos a la carga con una aplicación comercial de las que tanto nos gustan, en este caso se trata de PDF password Remover 2.2, de los últimos mensajes de la lista se pedía este programa para evitar los passwords de los archivos PDF's y creo que fue mi amigo k0rt el que pasó el link, yo lo vi ya que estaba por ahí, jeje.. y le eché un vistazo, mi sorpresa al ver que era un Trial de los que a nosotros nos gustan y ya estamos acostumbrados así que dije a este le haremos un tute ;-)

Lo primero de todo lógicamente es descargar e instalar el programa en cuestión, lo pondremos en "C:\Archivos de programa\PDF Password Remover v2.2" para tenerlo todos en el mismo sitio, luego abrimos su ejecutable winDecrypt.exe en el PEiD a ver que dice:

```
UPX 0.89.6 - 1.02 / 1.05 - 1.24 -> Markus & Laszlo
```

Es un UPX de los packers más sencillitos que podemos encontrarnos. El tema del desempakado no esconde ningún secreto: no esconde el OEP, no redirecciona la iat, no tiene ningún método antidebug, etc., o sea que el packer casi que se le cae sólo, de todas formas lo haremos manualmente con Olly, cargamos el .exe y estando parados en el EntryPoint vemos:

```
004126F0   60              PUSHAD
004126F1   BE 00E04000     MOV ESI,winDecry.0040E000
004126F6   8DBE 0030FFFF   LEA EDI,DWORD PTR DS:[ESI+FFFF3000]
004126FC   57              PUSH EDI
004126FD   83CD FF         OR EBP,FFFFFFFF
00412700 v EB 10           JMP SHORT winDecry.00412712
00412702   90              NOP
00412703   90              NOP
00412704   90              NOP
00412705   90              NOP
00412706   90              NOP
00412707   90              NOP
00412708   8A06            MOV AL,BYTE PTR DS:[ESI]
0041270A   46              INC ESI
```

en el PUSHAD arranca el packer y empieza su trabajo, cuando lo termine ejecutará un POPAD que dejará los registros intactos, podemos usar el método del PUSHAD-POPAD para hallar el OEP pero es tan tan fácil que no hace ni falta, sólo tenemos que bajar con la ruedecilla del ratón hacia abajo y encontramos el salto al OEP, es este:

```
0041282F   55              PUSH EBP
00412830   FF96 E42C0100   CALL DWORD PTR DS:[ESI+12CE4]
00412836   09C0            OR EAX,EAX
00412838 v 74 07           JE SHORT winDecry.00412841
0041283A   8903            MOV DWORD PTR DS:[EBX],EAX
0041283C   83C3 04         ADD EBX,4
0041283F ^ EB D8           JMP SHORT winDecry.00412819
00412841   FF96 E82C0100   CALL DWORD PTR DS:[ESI+12CE8]
00412847   61              POPAD
00412848 - E9 BB40FFFF     JMP winDecry.00406908   <-- Salto al OEP..!!
0041284D   0000            ADD BYTE PTR DS:[EAX],AL
0041284F   0000            ADD BYTE PTR DS:[EAX],AL
00412851   0000            ADD BYTE PTR DS:[EAX],AL
00412853   0000            ADD BYTE PTR DS:[EAX],AL
```

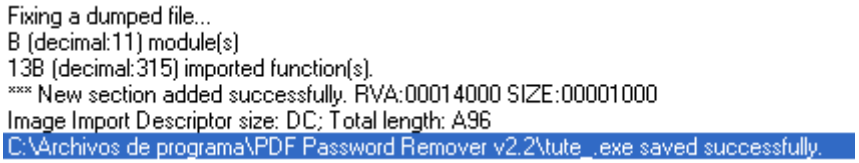
así que el OEP ya lo hemos encontrado está en :406908 donde apunta el salto, veamos que hay en esa posición de memoria, ponemos un BP en el salto y damos RUN, cuando pare lo ejecutamos con F7/F8 y vemos esto:

```
00406908   55              PUSH EBP
00406909   8BEC            MOV EBP,ESP
0040690B   6A FF           PUSH -1
0040690D   68 90914000     PUSH winDecry.00409190
00406912   68 6C6A4000     PUSH winDecry.00406A6C          JMP to msvcrt._except_handler3
00406917   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0040691D   50              PUSH EAX
0040691E   64:8925 000000  MOV DWORD PTR FS:[0],ESP
00406925   83EC 68         SUB ESP,68
00406928   53              PUSH EBX
```

Es el típico inicio de un programa compilado en C++ luego veremos de qué versión se trata, de momento dumpeamos desde aquí con la opción "Rebuild Import" desactivada como

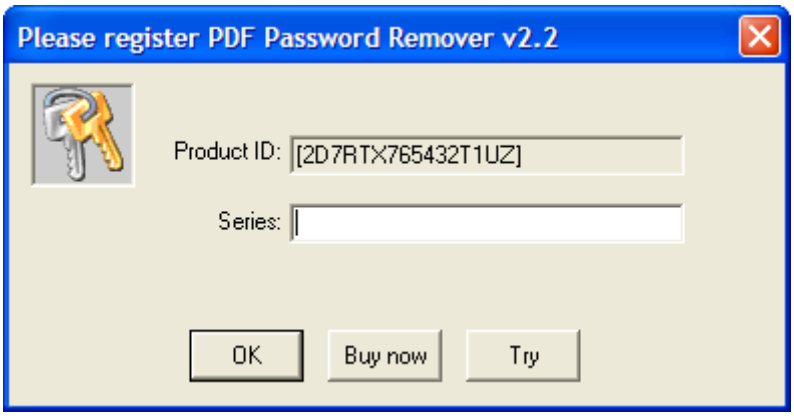
siempre hacemos, luego abrimos el Import Reconstructor y elegimos el proceso de la lista, le ponemos los valores correctos:

elegimos "Get Imports" y nos salen las funciones importadas, como he dicho antes no toca ni una así que está la tabla totalmente correcta, damos a "Fix Dump" y elegimos el archivo que acabamos de dumpear:

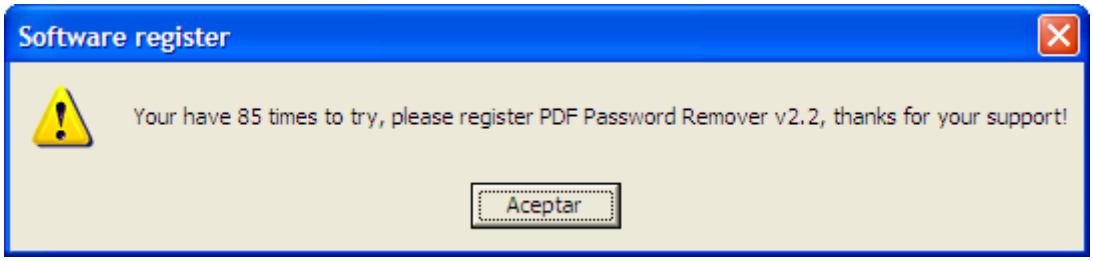


Allí nos dice que se creó correctamente el archivo tute_.exe (el dumpeado era tute.exe sin el guión bajo) bueno pues este archivo ya está desempakado ;-)) ya os dije que era muy pero que muy fácil de conseguir y veis que no os mentía.

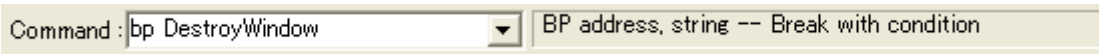
Ahora que lo tenemos así ya podemos meterle mano con el Olly e incluso parchear si hiciera falta pues los bytes en disco coincidirán con los bytes en memoria, si lanzamos el programa normalmente nos encontramos esta nagscreen:



Si pulsamos el botón Try le estamos diciendo que queremos probar el programa, pero todavía no nos dejan tranquilos nos informan de lo siguiente:



Pues eso nos dice el número de ejecuciones que nos quedan del período Trial, en este momento son 85, nos da 100 así que yo ya gasté 15, jeje.. bueno esto imagino que tendrá solución verdad? vamos a intentar arreglarlo, cerramos todos los mensajes, reiniciamos olly con Ctrl+F2 y damos RUN, aparecerá la primera nag que ya hemos visto en la imagen, antes de pulsar nada vamos a Olly y ponemos:



Esto hará que se detenga la ejecución cuando haga uso de esta APi, normalmente para cerrar ventanas y cuadros de diálogo los programas suelen llamarla bastante, en este caso también es llamada como se puede observar ya que Olly para aquí en la API:

Allí está al inicio de la api, esto no nos interesa, vamos retornando con Ctrl+F9 o bien con F8 hasta salir de la librería user.dll, estamos en esta parte:

```
00371A18   FF15 34333800   CALL DWORD PTR DS:[383334]          USER32.MessageBoxA
00371A1E   46              INC ESI
00371A1F   56              PUSH ESI
00371A20   E8 BBF8FFFF     CALL verypdf.003712E0
00371A25   83C4 04         ADD ESP,4
00371A28   C78424 E000000( MOV DWORD PTR SS:[ESP+E0],-1
00371A33   8D4C24 0C       LEA ECX,DWORD PTR SS:[ESP+C]
00371A37   E8 6DA90000     CALL verypdf.0037C3A9
00371A3C   68 15040000     PUSH 415
00371A41   57              PUSH EDI
00371A42   FF15 30333800   CALL DWORD PTR DS:[383330]          USER32.EndDialog
00371A48 v E9 CA000000     JMP verypdf.00371B17
00371A4D   8BB424 E800000( MOV ESI,DWORD PTR SS:[ESP+E8]
00371A54   68 10723800     PUSH verypdf.00387210               ASCII "Please register PDF Password Remover v2.2"
00371A59   56              PUSH ESI
00371A5A   FF15 58333800   CALL DWORD PTR DS:[383358]          USER32.SetWindowTextA
00371A60   68 60A83800     PUSH verypdf.0038A860               ASCII "2D7RTX765432T1UZ"
00371A65   E8 26FBFFFF     CALL verypdf.00371590
00371A6A   83C4 04         ADD ESP,4
00371A6D   8D4C24 10       LEA ECX,DWORD PTR SS:[ESP+10]
00371A71   C605 70A83800 ( MOV BYTE PTR DS:[38A870],0
00371A78   68 60A83800     PUSH verypdf.0038A860               ASCII "2D7RTX765432T1UZ"
00371A7D   68 08723800     PUSH verypdf.00387208               ASCII "[%s]"
00371A82   51              PUSH ECX
00371A83   E8 F20C0000     CALL verypdf.0037277A
00371A88   8B3D 5C333800   MOV EDI,DWORD PTR DS:[38335C]       USER32.SetDlgItemTextA
00371A8E   83C4 0C         ADD ESP,0C
00371A91   8D5424 10       LEA EDX,DWORD PTR SS:[ESP+10]
00371A95   52              PUSH EDX
00371A96   68 FE030000     PUSH 3FE
00371A9B   56              PUSH ESI
00371A9C   FFD7            CALL EDI
00371A9E   E8 7DF9FFFF     CALL verypdf.#2
00371AA3   85C0            TEST EAX,EAX
00371AA5 v 75 15           JNZ SHORT verypdf.00371ABC
00371AA7   6A 01           PUSH 1
```

Asoma una referencia a MessageBoxA encargada de mostrarnos el segundo mensaje en el que nos informa del número de ejecuciones restantes, esto no es todo, si seguimos traceando superando el siguiente RET caemos nuevamente en user.dll, debemos ir traceando con Ctrl+F9 superando RET's, pasaremos por la MFC42.dll ni caso, seguimos hasta llegar a:

```
00371B5C   51              PUSH ECX
00371B5D   68 A3000000     PUSH 0A3
00371B62   50              PUSH EAX
00371B63   FF15 28333800   CALL DWORD PTR DS:[383328]          USER32.DialogBoxParamA
00371B69   A1 28A93800     MOV EAX,DWORD PTR DS:[38A928]
00371B6E   C2 0800         RETN 8
00371B71   90              NOP
00371B72   90              NOP
00371B73   90              NOP
```

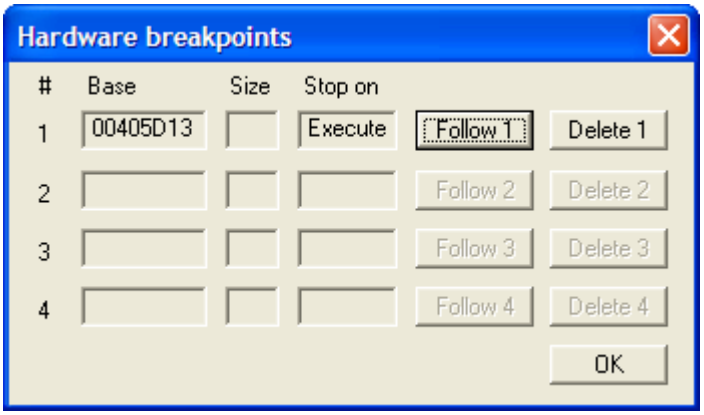
Vemos la referencia a DialogBoxParamA que es la responsable de la nag que nos sacó al inicio, el siguiente RET nos pone en el código del ejecutable que estamos estudiando:

```
00405D1C   C705 E4B44000 ( MOV DWORD PTR DS:[40B4E4],0
00405D26   FF15 80844000   CALL DWORD PTR DS:[<&user32.GetDesktopW USER32.GetDesktopWindow
00405D2C   50              PUSH EAX
00405D2D   A1 E4B44000     MOV EAX,DWORD PTR DS:[40B4E4]
00405D32   50              PUSH EAX
00405D33   E8 F8050000     CALL <JMP.&verypdf.#1>
00405D38   A3 E4B44000     MOV DWORD PTR DS:[40B4E4],EAX
00405D3D   A1 E4B44000     MOV EAX,DWORD PTR DS:[40B4E4]
00405D42   85C0            TEST EAX,EAX
00405D44 v 0F84 92000000   JE dump_.00405DDC
00405D4A   8D4C24 08       LEA ECX,DWORD PTR SS:[ESP+8]
00405D4E   68 04010000     PUSH 104
```

Ese salto a verypdf.dll es el que nos pone en dicha librería donde nos acribillan a mensajes y demás, si miramos por encima del CALL vemos esto:

```
00405D06   C705 E4B44000 ( MOV DWORD PTR DS:[40B4E4],1
00405D10   56              PUSH ESI
00405D11   8BF1            MOV ESI,ECX
00405D13   E8 1E060000     CALL <JMP.&verypdf.#2>
00405D18   85C0            TEST EAX,EAX
00405D1A v 74 21           JE SHORT tute_.00405D3D
00405D1C   C705 E4B44000 ( MOV DWORD PTR DS:[40B4E4],0
00405D26   FF15 80844000   CALL DWORD PTR DS:[<&user32.GetDesktopW USER32.GetDesktopWindow
00405D2C   50              PUSH EAX
00405D2D   A1 E4B44000     MOV EAX,DWORD PTR DS:[40B4E4]
00405D32   50              PUSH EAX
00405D33   E8 F8050000     CALL <JMP.&verypdf.#1>
00405D38   A3 E4B44000     MOV DWORD PTR DS:[40B4E4],EAX
00405D3D   A1 E4B44000     MOV EAX,DWORD PTR DS:[40B4E4]
00405D42   85C0            TEST EAX,EAX
00405D44 v 0F84 92000000   JE tute_.00405DDC
00405D4A   8D4C24 08       LEA ECX,DWORD PTR SS:[ESP+8]
```

Un CALL que apunta precisamente a veripdf.dll y un salto condicional detrás que en caso de producirse evitaría que saque la nagscreen y el mensaje recordatorio de ejecuciones, muy pero que muy interesante sí señor, ponemos un HBP en dicho CALL y reiniciamos olly:



Ahí lo tenemos puesto para que pare al pasar por este CALL, lanzamos la ejecución con F9 y Olly para en este punto, vamos a entrar en el CALL a ver que encontramos de interesante en él:

```
00371420   81EC 00040000   SUB ESP,400
00371426   8D4424 00       LEA EAX,DWORD PTR SS:[ESP]
0037142A   50              PUSH EAX
0037142B   E8 00FFFFFF     CALL verypdf.00371330
00371430   83C4 04         ADD ESP,4
00371433   85C0            TEST EAX,EAX
00371435 v 75 0C           JNZ SHORT verypdf.00371443
00371437   B8 01000000     MOV EAX,1
0037143C   81C4 00040000   ADD ESP,400
00371442   C3              RETN
00371443   8D4C24 00       LEA ECX,DWORD PTR SS:[ESP]
00371447   51              PUSH ECX
00371448   E8 33000000     CALL verypdf.00371480
0037144D   83C4 04         ADD ESP,4
00371450   85C0            TEST EAX,EAX
00371452 v 75 0C           JNZ SHORT verypdf.00371460
00371454   B8 02000000     MOV EAX,2
00371459   81C4 00040000   ADD ESP,400
0037145F   C3              RETN
00371460   8D9424 0401000( LEA EDX,DWORD PTR SS:[ESP+104]
00371467   52              PUSH EDX
00371468   E8 93FBFFFF     CALL verypdf.00371000
0037146D   83C4 04         ADD ESP,4
00371470   F7D8            NEG EAX
00371472   1BC0            SBB EAX,EAX
00371474   24 FD           AND AL,0FD
00371476   83C0 03         ADD EAX,3
00371479   81C4 00040000   ADD ESP,400
0037147F   C3              RETN
```

Este es el código de dicho CALL, como vemos no es demasiado, hay tres RET's con diferentes valores que puede tomar EAX en cada uno de ellos, a nosotros nos interesaría retornar con EAX==0 para que se ejecute el salto:

```
00405D1A   74 21            JE SHORT tute_.00405D3D
```

que evita que se visualice la nag, al final de la rutina vemos un "ADD EAX,3" que le suma 3 a este registro, así que para salir con EAX==0 deberá valer -3 en ese instante, de momento traceemos el CALL a ver que vemos:

```
0037133B   56              PUSH ESI
0037133C   57              PUSH EDI
0037133D   68 04010000     PUSH 104
00371342   50              PUSH EAX
00371343   FF15 74313800   CALL DWORD PTR DS:[383174]        kernel32.GetSystemDirectoryA
00371349   8D7C24 0C       LEA EDI,DWORD PTR SS:[ESP+C]
0037134D   83C9 FF         OR ECX,FFFFFFFF
00371350   33C0            XOR EAX,EAX
00371352   F2:AE           REPNE SCAS BYTE PTR ES:[EDI]
00371354   F7D1            NOT ECX
```

Obtiene el directorio de sistema, o sea donde tenemos instalado el window$, en mi caso la api retorna en [ESP+C] el path a "C:\windows\system32" y ahí no queda la cosa:

```
0037138A   F3:A4           REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:
0037138C   BF A0713800     MOV EDI,verypdf.003871A0         ASCII "pwdremover.dat"
00371391   83C9 FF         OR ECX,FFFFFFFF
00371394   33C0            XOR EAX,EAX
00371396   8D5424 0C       LEA EDX,DWORD PTR SS:[ESP+C]
0037139A   F2:AE           REPNE SCAS BYTE PTR ES:[EDI]
```

lo que va a hacer es evidente, se mete en el directorio de sistema donde busca el archivo "pwdremover.dat" jeje, más que significativo esto, además le siguen llamadas a OpenFile, _lopen, _lread y _lclose así que está claro que va a acceder al mismo y a leer su contenido, si miramos éste desde un editor hexa tipo UEdit32 veremos que sólo almacena un número ¿el de ejecuciones quizá? después de esto sale del CALL con EAX==1, hemos de recordar que estamos estudiando el CALL siguiente:

```
00405D13    E8 1E060000     CALL <JMP.&verypdf.#2>
```

y que debe retornar EAX==0 ok? dentro del mismo encontramos 3 subcall, pues bien el que se encarga de leer el contenido del archivo "pwdremover.dat" es el primero de ellos, como salimos con EAX==1 salta y evita el "MOV EAX,1" y el RET que nos saca:

```
0037142B    E8 00FFFFFF     CALL verypdf.00371330
00371430    83C4 04         ADD ESP,4
00371433    85C0            TEST EAX,EAX
00371435    75 0C           JNZ SHORT verypdf.00371443
00371437    B8 01000000     MOV EAX,1
0037143C    81C4 00040000   ADD ESP,400
00371442    C3              RETN
```

Esto quiere decir que saltamos en :371435 y llegamos a :371443 donde encontramos el segundo CALL:

```
00371448    E8 33000000     CALL verypdf.00371480
0037144D    83C4 04         ADD ESP,4
00371450    85C0            TEST EAX,EAX
00371452    75 0C           JNZ SHORT verypdf.00371460
00371454    B8 02000000     MOV EAX,2
00371459    81C4 00040000   ADD ESP,400
0037145F    C3              RETN
```

Vemos que el código es similar sólo que ahora es un "MOV EAX,2" es evidente que deberemos saltar en :371452 para evitarlo (recordar que es EAX==0 el retorno válido) así que entraremos en el CALL a ver que ocurre dentro, pues parece que está revisando el contenido del archivo "pwdremover.dat" y como no le gusta nos saca fuera rápidamente con EAX==0 lo que hará que no salte en :37142 y quede un retorno final de EAX==2 que indicará que algo no fue bien ;(

Podríamos parchear el código original de la dll por este otro:

```
00371472   1BC0          SBB EAX,EAX
00371474   24 FD         AND AL,0FD
00371476   90            NOP
00371477   90            NOP
00371478   90            NOP
00371479   81C4 00040000 ADD ESP,400
0037147F   C3            RETN
00371480   53            PUSH EBX
00371481   55            PUSH EBP
00371482   8B6C24 0C     MOV EBP,DWORD PTR SS:[ESP+C]
00371486   56            PUSH ESI
00371487   57            PUSH EDI
00371488   B8 01000000   MOV EAX,1
0037148D   33C9          XOR ECX,ECX
0037148F   BF 00020000   MOV EDI,200
00371494   8D3429        LEA ESI,DWORD PTR DS:[ECX+EBP]
00371497   8AD1          MOV DL,CL
00371499   F6D2          NOT DL
0037149B   8A1C37        MOV BL,BYTE PTR DS:[EDI+ESI]
0037149E   32DA          XOR BL,DL
003714A0   8A16          MOV DL,BYTE PTR DS:[ESI]
003714A2   3ADA          CMP BL,DL
003714A4   90            NOP
003714A5   90            NOP
003714A6   41            INC ECX
003714A7   81F9 00020000 CMP ECX,200
003714AD ^ 7C E5         JL SHORT verypdf.00371494
003714AF   5F            POP EDI
003714B0   5E            POP ESI
003714B1   5D            POP EBP
003714B2   5B            POP EBX
003714B3   C3            RETN
003714B4   5F            POP EDI
003714B5   5E            POP ESI
003714B6   5D            POP EBP
003714B7   33C0          XOR EAX,EAX
003714B9   5B            POP EBX
003714BA   C3            RETN
```

Esto aseguraría que el CALL retornara siempre EAX==1 que es lo que nos interesa en este subcall, luego además nopeamos el "ADD EAX,3" que no nos gusta nada, miramos y nos damos cuenta de que llegamos con EAX==0 justo antes de sumarle las 3 unidades así que con el nopeo quedará este valor que ya nos va bien como retorno de la función. Perfecto, vamos a parchear...

UPX 0.80 - 1.24 DLL -> Markus & Laszlo

pues vaya la dll también está empakada con UPX, no sería gran rival ya que es muy sencillito y aprendimos a desempakar dlls con [Juan Jose] un saludo amigo ;-) pero no obstante vamos a ser más eficientes, probaremos poner un BP común en la primera instrucción del CALL que queremos parchear, o sea aquí:

```
00371420 >  81EC 00040000   SUB ESP,400                    ;Aquí el BP..!!
00371426    8D4424 00       LEA EAX,DWORD PTR SS:[ESP]
```

Y dejar correr la aplicación, no sin antes reiniciar Olly con Ctrl+F2, pues bien nos damos cuenta de que sólo se pasa una vez por este CALL lo que nos da la posibilidad de parchear el .exe ya desempakado con sólo falsear el valor de retorno en EAX o incluso el salto en:

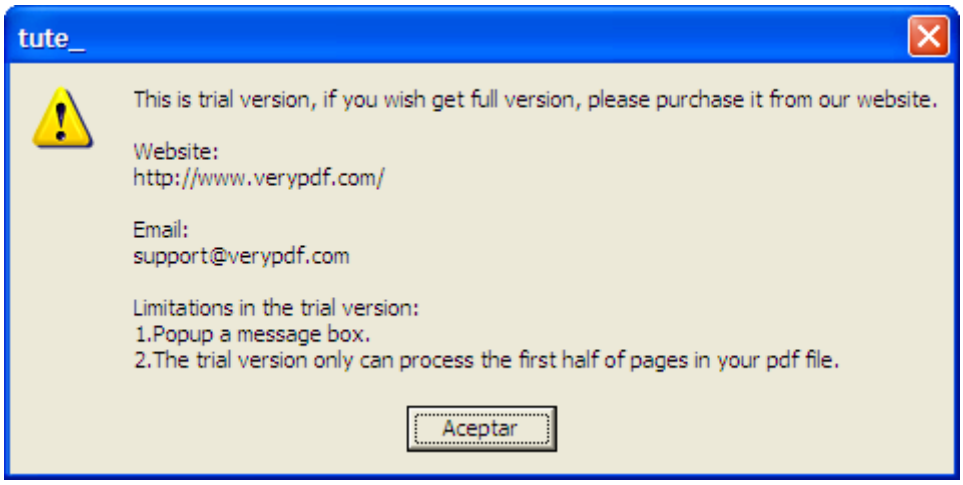
```
00405D13    E8 1E060000     CALL <JMP.&verypdf.#2>
00405D18    85C0            TEST EAX,EAX
00405D1A    74 21           JE SHORT tute_.00405D3D
```

que es donde se llama a toda la rutina de comprobación, esto estaría bien porque nos ahorraría el trabajo de desempakado de la librería que aunque sencillo no es menos trabajo, entonces podríamos comenzar cambiando la zona a anterior por esta otra:

```
00405D06   C705 E4B44000  MOV DWORD PTR DS:[40B4E4],1
00405D10   56             PUSH ESI
00405D11   8BF1           MOV ESI,ECX
00405D13   E8 1E060000    CALL <JMP.&verypdf.#2>
00405D18   33C0           XOR EAX,EAX
00405D1A   EB 21          JMP SHORT tute_.00405D3D
00405D1C   C705 E4B44000  MOV DWORD PTR DS:[40B4E4],0
00405D26   FF15 80844000  CALL DWORD PTR DS:[<&user32.GetDesktopW  USER32.GetDesktopWindow
00405D2C   50             PUSH EAX
```

Esto lo hacemos repito porque sólo hemos visto una llamada a la rutina, si hubiera más sería absurdo hacer esto porque las demás llamadas retornarían un valor erróneo y deberíamos parchear cada vez el retorno llegando a ser cansado también de hacer.

Guardamos los cambios y lo salvamos como tute_1.exe por ejemplo, lo ejecutamos y entramos sin la nag, jeje.. vamos muy bien tampoco aparece el mensaje informativo del número de ejecuciones ni tampoco al abrir un PDF aparece este mensaje:



Que por su puesto sí aparecía en la versión Trial, esto quiere decir que el programa se comporta como si estuviera registrado ;-) y encima evitamos meternos con la dll, matamos dos pájaros de un tiro entonces, el botón Register también ha desaparecido del cuadro principal así que todo apunta a que el parche es bueno, resta decir que el programa queda totalmente "registrado" de esta manera, eso sí no hay nada como encontrar un serial válido para el programa en cuestión así que vamos a ello...

Lo primero que haremos será localizar y aislar la rutina de verificación del serial, mucho nos puede ayudar en este caso que tenga mensaje de error al introducir un serial no_válido, miraremos si este muestra alguno, introducimos nuestro nombre seguido de un serial cualquiera pulsamos "OK" y bouuuuuuuuum...

Pues sí que tiene mensaje de error, por aquí atacaremos, pongamos un BP primero:

```
Command : bp MessageBoxA
```

Ponemos este BreakPoint antes de reintroducir el serial, cuando lo hayamos puesto entonces sí nos vamos al cuadro de registro y aceptamos, automáticamente olly para en user32.dll lo que indica que se está haciendo uso de esta api, retornamos con Ctrl+F9 y el mensaje aparecerá en pantalla, lo aceptamos y retornamos a Olly nuevamente pero mirad donde caemos es una zona muy interesante:

```
00371847   F3:AB          REP STOS DWORD PTR ES:[EDI]
00371849   FF15 2C333800  CALL DWORD PTR DS:[38332C]      USER32.GetDlgItemTextA
0037184F   68 60A83800    PUSH verypdf.0038A860           ASCII "12121212"
00371854   E8 A7F7FFFF    CALL verypdf.00371000
00371859   83C4 04        ADD ESP,4
0037185C   85C0           TEST EAX,EAX
0037185E   74 39          JE SHORT verypdf.00371899
00371860   6A 40          PUSH 40
00371862   68 70733800    PUSH verypdf.00387370           ASCII "Thank you purchased the"
00371867   68 3C733800    PUSH verypdf.0038733C           ASCII "Thank you purchased the PDF Password Remover v2.2."
0037186C   56             PUSH ESI
0037186D   FF15 34333800  CALL DWORD PTR DS:[383334]      USER32.MessageBoxA
00371873   68 60A83800    PUSH verypdf.0038A860           ASCII "12121212"
00371878   56             PUSH ESI
00371879   E8 F2F8FFFF    CALL verypdf.00371170
0037187E   83C4 08        ADD ESP,8
00371881   C705 28A93800  MOV DWORD PTR DS:[38A928],1
0037188B   6A 01          PUSH 1
0037188D   56             PUSH ESI
0037188E   FF15 30333800  CALL DWORD PTR DS:[383330]      USER32.EndDialog
00371894   E9 7E020000    JMP verypdf.00371B17
00371899   6A 10          PUSH 10
0037189B   6A 00          PUSH 0
0037189D   68 08733800    PUSH verypdf.00387308           ASCII "Series number error, please check it and try again."
003718A2   56             PUSH ESI
003718A3   FF15 34333800  CALL DWORD PTR DS:[383334]      USER32.MessageBoxA
003718A9   68 FB030000    PUSH 3FB
003718AE   56             PUSH ESI
003718AF   FF15 38333800  CALL DWORD PTR DS:[383338]      USER32.GetDlgItem
003718B5   50             PUSH EAX
003718B6   FF15 54333800  CALL DWORD PTR DS:[383354]      USER32.SetFocus
003718BC   E9 56020000    JMP verypdf.00371B17
003718C1   6A 01          PUSH 1
003718C3   6A 00          PUSH 0
```

Vemos una llamada inicial a GetDlgItemTextA donde recoge el serial del cuadro de registro, poco después aparecen ambas cadenas la de error y la de felicitación, lo que hace que se visualice una u otra es esta CALL:

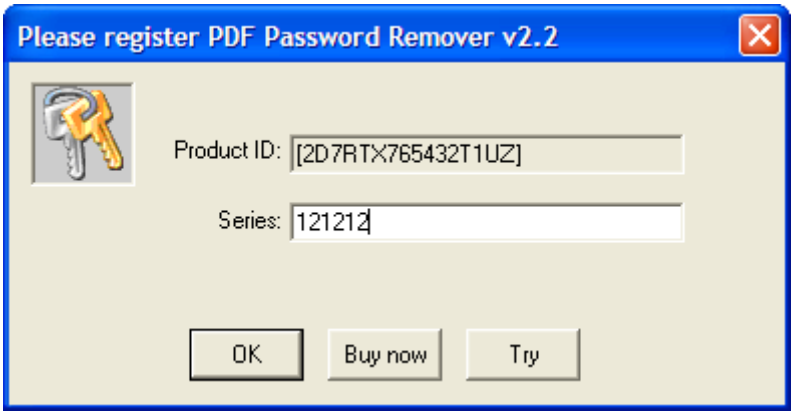
```
00371854    E8 A7F7FFFF     CALL verypdf.00371000
00371859    83C4 04         ADD ESP,4
0037185C    85C0            TEST EAX,EAX
0037185E    74 39           JE SHORT verypdf.00371899
```

Ese salto condicional JE es muy importante porque de él depende el mensaje que veremos (error o felicitación) este salto está controlado por el CALL que tiene justo delante y que no es más que una llamada a la rutina de verificación, ya la tenemos localizada y aislada, ahora la estudiaremos a ver si se le puede sacar un serial válido, estudiaremos el algoritmo eso sí por encima para ver como trabaja y que restricciones tiene, pondremos un BP en :371854 para comenzar a estudiar el CALL_verificador del serial que es precisamente el de esta posición de memoria, una vez paremos por el BP entramos con F7 y a tracear se ha dicho atentos a todo lo que veamos, para tenerlo más claro aportaré una imagen donde recoge el código del CALL_verificador donde estamos traceando ahora, como se ve no es mucha cosa, un par de Subcalls donde hace algunas cosillas y tres salidas distintas dos con EAX==0 (flag de error) y la tercera que pone un valor en AL (0 ó 1) en función del resultado de la comparación entre CL y 26h, con esto ya tenemos el esquema principal de la rutina, ahora como digo la tracearemos en busca de datos más tangibles con los que trabajaremos e iremos formando nuestro serial válido, de momento como digo aportaremos una imagen que vale más que mil palabras ;-)

/ver pág. siguiente/

```
00371000   83EC 18        SUB ESP,18                      inicio rutina verificacion
00371003   53              PUSH EBX
00371004   56              PUSH ESI
00371005   8B7424 24       MOV ESI,DWORD PTR SS:[ESP+24]
00371009   8D5424 08       LEA EDX,DWORD PTR SS:[ESP+8]
0037100D   57              PUSH EDI
0037100E   32DB            XOR BL,BL
00371010   8A46 0E         MOV AL,BYTE PTR DS:[ESI+E]
00371013   8A4E 0F         MOV CL,BYTE PTR DS:[ESI+F]
00371016   52              PUSH EDX
00371017   884424 1C       MOV BYTE PTR SS:[ESP+1C],AL
0037101B   885C24 1D       MOV BYTE PTR SS:[ESP+1D],BL
0037101F   884C24 10       MOV BYTE PTR SS:[ESP+10],CL
00371023   885C24 11       MOV BYTE PTR SS:[ESP+11],BL
00371027   E8 49150000     CALL verypdf.00372575
0037102C   8BF8            MOV EDI,EAX
0037102E   8D4424 1C       LEA EAX,DWORD PTR SS:[ESP+1C]
00371032   50              PUSH EAX
00371033   E8 3D150000     CALL verypdf.00372575           subcall_verificador
00371038   03F8            ADD EDI,EAX
0037103A   83C4 08         ADD ESP,8
0037103D   83FF 0B         CMP EDI,0B                      debe quedar EDI==0Bh
00371040 v 74 09           JE SHORT verypdf.0037104B
00371042   5F              POP EDI
00371043   5E              POP ESI
00371044   33C0            XOR EAX,EAX                     flag de error activado..!!
00371046   5B              POP EBX
00371047   83C4 18         ADD ESP,18
0037104A   C3              RETN
0037104B   8A0E            MOV CL,BYTE PTR DS:[ESI]
0037104D   8A56 01         MOV DL,BYTE PTR DS:[ESI+1]
00371050   8D4424 0C       LEA EAX,DWORD PTR SS:[ESP+C]
00371054   884C24 18       MOV BYTE PTR SS:[ESP+18],CL
00371058   50              PUSH EAX
00371059   885C24 1D       MOV BYTE PTR SS:[ESP+1D],BL
0037105D   885424 10       MOV BYTE PTR SS:[ESP+10],DL
00371061   885C24 11       MOV BYTE PTR SS:[ESP+11],BL
00371065   E8 0B150000     CALL verypdf.00372575
0037106A   8D4C24 1C       LEA ECX,DWORD PTR SS:[ESP+1C]
0037106E   8BF8            MOV EDI,EAX
00371070   51              PUSH ECX
00371071   E8 FF140000     CALL verypdf.00372575           subcall_verificador
00371076   03F8            ADD EDI,EAX
00371078   83C4 08         ADD ESP,8
0037107B   83FF 09         CMP EDI,9                       debe quedar EDI==09h
0037107E v 74 09           JE SHORT verypdf.00371089
00371080   5F              POP EDI
00371081   5E              POP ESI
00371082   33C0            XOR EAX,EAX                     flag de error activado..!!
00371084   5B              POP EBX
00371085   83C4 18         ADD ESP,18
00371088   C3              RETN
00371089   8A4E 05         MOV CL,BYTE PTR DS:[ESI+5]
0037108C   33C0            XOR EAX,EAX
0037108E   80F9 26         CMP CL,26                       debe ser CL==26h
00371091   5F              POP EDI
00371092   5E              POP ESI
00371093   5B              POP EBX
00371094   0F94C0          SETE AL                         tiene que quedar AL==1..!!
00371097   83C4 18         ADD ESP,18
0037109A   C3              RETN
0037109B   90              NOP
0037109C   90              NOP
0037109D   90              NOP
```

Ahí está a grandes rasgos detallado lo que hace la rutina, los puntos calientes están en :371040 , :37107B y :371008E, debemos superar todas estas comprobaciones para llegar a la comparación entre CL y 26h con opciones, así que introducimos un serial y a trabajar se ha dicho, pondremos de momento este:



Y aceptamos pulsando el botón "OK" Olly salta en el CALL_verificador debido al BP que antes le pusimos:

```
00371854   E8 A7F7FFFF     CALL verypdf.00371000
00371859   83C4 04         ADD ESP,4
0037185C   85C0            TEST EAX,EAX
0037185E v 74 39           JE SHORT verypdf.00371899
```

entramos con F7 y a tracear con cuidado, llegamos al primer punto clave:

```
00371033    E8 3D150000     CALL verypdf.00372575            ; subcall_verificador
00371038    03F8            ADD EDI,EAX
0037103A    83C4 08         ADD ESP,8
0037103D    83FF 0B         CMP EDI,0B                       ; debe quedar EDI==0Bh
00371040    74 09           JE SHORT verypdf.0037104B
```

Superamos el CALL y vemos que nos dejó EAX==0 ¿? quizá no le gustó la longitud del serial, repetimos la jugada pero con un serial más grande por ejemplo 121212-121212 pero sigue siendo lo mismo, lo pondremos todavía más grande 121212-121212-121212 y nos deja EAX==1, ya va cambiando algo, entramos en el subcall para ver de donde saca este valor y así entendamos quizá que es lo que debemos poner, el valor lo lee aquí:

```
00372519    85C0            TEST EAX,EAX
0037251B  v 74 03           JE SHORT verypdf.00372520
0037251D    47              INC EDI
0037251E  ^ EB D2           JMP SHORT verypdf.003724F2
00372520    0FB637          MOVZX ESI,BYTE PTR DS:[EDI]
00372523    47              INC EDI
00372524    83FE 2D         CMP ESI,2D
00372527    8BEE            MOV EBP,ESI
00372529  v 74 05           JE SHORT verypdf.00372530
```

De ahí obtiene el 1 que luego pasa a EAX y que a la postre será el retorno de la función, si hacemos un "Follow in Dump->Memory Address" descubrimos de donde lo lee:

```
Address   Hex dump                                          ASCII
0012EA68  31 00 37 00 98 06 0E 00 FB 03 00 00 59 18 37 00  1.7.ÿ♦Â.¹♥..Y↑7.
0012EA78  60 A8 38 00 D0 EB 12 00 40 17 37 00 00 FE F5 72  ¨8.Ðë↨.@↨7..■⌠
0012EA88  00 00 00 00 00 00 00 00 10 00 F6 72 FE 05 0F 00  ........►.÷r■♣☼.
0012EA98  00 00 00 00 CD AB BA DC 00 00 00 00 02 00 00 00  ....=½‖▄....☻...
0012EAA8  83 0C 01 05 DC EA 12 00 C7 1F F5 72 13 00 00 00  â.♀♪▬Ü↨.Ä▼⌠r‼...
```

Ahí lo tiene guardado en [12EA68] y es de donde lo va leyendo pero ¿quién lo pone en esta posición de memoria? poniendo un BPM en la zona lo descubrimos también:

```
0037100D    57              PUSH EDI
0037100E    32DB            XOR BL,BL
00371010    8A46 0E         MOV AL,BYTE PTR DS:[ESI+E]       de aqui lo lee AL..!!
00371013    8A4E 0F         MOV CL,BYTE PTR DS:[ESI+F]
00371016    52              PUSH EDX
00371017    884424 1C       MOV BYTE PTR SS:[ESP+1C],AL      modifica [12EA68]
0037101B    885C24 1D       MOV BYTE PTR SS:[ESP+1D],BL
0037101F    884C24 10       MOV BYTE PTR SS:[ESP+10],CL
00371023    885C24 11       MOV BYTE PTR SS:[ESP+11],BL
00371027    E8 49150000     CALL verypdf.00372575
0037102C    8BF8            MOV EDI,EAX
0037102E    8D4424 1C       LEA EAX,DWORD PTR SS:[ESP+1C]
00371032    50              PUSH EAX
00371033    E8 3D150000     CALL verypdf.00372575            subcall_verificador
00371038    03F8            ADD EDI,EAX
0037103A    83C4 08         ADD ESP,8
0037103D    83FF 0B         CMP EDI,0B                       debe quedar EDI==0Bh
00371040  v 74 09           JE SHORT verypdf.0037104B
00371042    5F              POP EDI
```

Vemos que lo rellena poco antes del primer subcall_verificador y lo más interesante es que AL toma este valor justo unas líneas por encima de [ESI+E] ponemos un BP en esta instrucción y cuando pare hacemos un "Follow in dump->Memory Address" para ver de donde lo está leyendo, nos encontramos con esto:

```
Address   Hex dump                                          ASCII
0038A85E  00 00 31 32 31 32 31 32 2D 31 32 31 32 31 32 2D  ..121212-121212-
0038A86E  31 32 31 32 31 32 00 00 00 00 00 00 00 00 00 00  121212..........
0038A87E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0038A88E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0038A89E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

el byte que remarqué en amarillo es el que está leyendo y corresponde con el 15° dígito de nuestro serial, por lo que parece que el EAX==1 lo saca de esta posición, para estar seguros de ello modificaremos esta posición del serial y miraremos el valor de EAX a la salida del primer subcall_verificador, ponemos como serial 121212-121212-621212:

```
00371032    50              PUSH EAX
00371033    E8 3D150000     CALL verypdf.00372575            subcall_verificador
00371038    03F8            ADD EDI,EAX
0037103A    83C4 08         ADD ESP,8
0037103D    83FF 0B         CMP EDI,0B                       debe quedar EDI==0Bh
```

veamos cuanto vale EAX:

```
Registers (MMX)
EAX 00000006
ECX 0012EA68
EDX 0012EA5C
EBX 00000000
ESP 0012EA48
EBP 0012EB88
ESI 0038A860 ASCII "121212-121212-621212"
EDI 00000002
```

jeje, parece que le hemos pillado verdad? esta posición del serial parece que influye directamente en el valor de retorno de este Subcall, pero para obtener EAX==0Bh que deberemos poner un 11 ocupará dos posiciones y no una ¿? este es el enigma a desvelar, si le pongo la "B" como si fuera hexadecimal retorna EAX==0 así que la clave está en buscar la manera de que retorne EAX==0Bh sabiendo que el 15° dígito influye directamente en este resultado, quizá haga una suma entre dos posiciones ¿? esto lo iremos viendo sobre la marcha porque yo todavía no lo he resuelto lo hago a medida que escribo así que veremos que es, de momento fijémonos bien en una cosa que se nos está pasando por alto:

```
00371027    E8 49150000     CALL verypdf.00372575          ;Ojito a este CALL
0037102C    8BF8            MOV EDI,EAX                    ;EDI toma el valor de EAX
0037102E    8D4424 1C       LEA EAX,DWORD PTR SS:[ESP+1C]
00371032    50              PUSH EAX
00371033    E8 3D150000     CALL verypdf.00372575          ;subcall_verificador
00371038    03F8            ADD EDI,EAX                    ; suma a EDI el valor de EAX
0037103A    83C4 08         ADD ESP,8
0037103D    83FF 0B         CMP EDI,0B                     ;debe quedar EDI==0Bh
00371040    74 09           JE SHORT verypdf.0037104B      ;tiene que saltar aquí..!!
```

Esto es!!! lo tenemos delante y no nos hemos fijado, mirad el CALL que hay en :371027 ¿que creéis que hace? pues dejar en EAX el valor del dígito que hay en la 16ª posición del serial, jeje.. y este valor lo pasa a EDI que se lo guarda, veamos un ejemplo:

Serial: 121212-121212-931212

el que remarco en amarillo es el dígito que va a leer en este CALL, ponemos un BP en él y lo pasamos con F8, a la salida miramos los registros (EAX en particular)

```
Registers (MMX)
EAX 00000003
ECX 0012EA5C
EDX 0012EA5C
EBX 00000000
ESP 0012EA4C
EBP 0012EB88
ESI 0038A860 ASCII "121212-121212-931212"
EDI 0038A928 verypdf.0038A928
```

jejeje.. vemos como quedó EAX==3 que es el valor del 16° dígito de nuestro serial, este valor lo almacena en EDI y no lo pierde, luego se mete en el CALL de :371033 donde pone en EAX el valor del 15° dígito y se lo suma a EDI que almacena el valor del 16°, el resultado lo compara con 0Bh i sea que lo que está mirando es que:

dígito_15 + dígito_16 = 0Bh (11 decimal)

hagamos la prueba nosotros mismos, vamos a poner dos valores que sumados den 11d por ejemplo el 9 y el 2 e irán en las posiciones 15 y 16 respectivamente en nuestro serial:

Serial: 121212-121212-921212

estos dos dígitos ya suman 11, introducimos el key y aceptamos, la salida del primer CALL deja EAX==2 perfecto, pasa a EDI este valor que lo guarda, en el siguiente CALL deja EAX==9 jeje.. y suma este valor a EDI dejando lógicamente EDI==0Bh y salta ;-)

```
00371032   50            PUSH EAX
00371033   E8 3D150000   CALL verypdf.00372575        subcall_verificador
00371038   03F8          ADD EDI,EAX
0037103A   83C4 08       ADD ESP,8
0037103D   83FF 0B       CMP EDI,0B                   debe quedar EDI==0Bh
00371040   74 09         JE SHORT verypdf.0037104B
00371042   5F            POP EDI
00371043   5E            POP ESI
00371044   33C0          XOR EAX,EAX                  flag de error activado..!!
00371046   5B            POP EBX
00371047   83C4 18       ADD ESP,18
0037104A   C3            RETN
0037104B   8A0E          MOV CL,BYTE PTR DS:[ESI]
0037104D   8A56 01       MOV DL,BYTE PTR DS:[ESI+1]
```

Ya tenemos pasada la primera restricción, quedan más.. vemos que se mete en un CALL poco después y compara EDI con 9 si no es así adiós:

```
00371071   E8 FF140000   CALL verypdf.00372575        subcall_verificador_2
00371076   03F8          ADD EDI,EAX
00371078   83C4 08       ADD ESP,8
0037107B   83FF 09       CMP EDI,9                    debe quedar EDI==09h
0037107E   74 09         JE SHORT verypdf.00371089
```

Esto es otra restricción que deberemos pasar, debemos encontrar lo que está verificando, con el mismo serial de antes 121212-121212-921212 veremos que valor de retorno deja en al registro EAX a la salida del CALL de :371071 y así nos haremos una idea de lo que está buscando y donde lo está haciendo claro, lo que busca sabemos que es un 9, veamos:

```
Registers (MMX)
EAX 00000001
ECX 0012EC18
EDX 0012EC32
EBX 00000000
ESP 0012EBF8
EBP 0012ED38
ESI 0038A860 ASCII "121212-121212-921212"
EDI 00000002
```

un 1, pero en nuestro serial hay demasiados unos como para saber a qué posición se refiere, lo que haremos es ir variando los unos por otros valores hasta encontrar justo la posición que mira, pues mira por donde que el primero que cambio es el que busca, jeje.. en este caso es el primer dígito y quiere un 9 allí, cambiamos el serial por:

921212-121212-921212

En amarillo las posiciones que va verificando, con este serial veamos si pasa la restricción ahora poniendo un BP en el segundo salto, no se produce porque EDI==0Bh, claro hombre está mirando otra pareja de dígitos, creo que son el 1° y el 2° que suman 11 justo el valor de EDI, reajustemos la pareja de manera que sumen 9 ambos dígitos:

721212-121212-921212

Y probamos con él:

```
00371071   E8 FF140000   CALL verypdf.00372575          subcall_verificador_2
00371076   03F8          ADD EDI,EAX
00371078   83C4 08       ADD ESP,8
0037107B   83FF 09       CMP EDI,9                      debe quedar EDI==09h
0037107E   74 09         JE SHORT verypdf.00371089
00371080   5F            POP EDI
00371081   5E            POP ESI
00371082   33C0          XOR EAX,EAX                    flag de error activado..!!
00371084   5B            POP EBX
00371085   83C4 18       ADD ESP,18
00371088   C3            RETN
00371089   8A4E 05       MOV CL,BYTE PTR DS:[ESI+5]
```

ahora sí que salta, ha sumado los dos primeros dígitos y el resultado debe ser igual a 9, esta es la segunda restricción del serial:

dígito_1 + dígito_2 = 09h (9 decimal)

Y nos queda una última restricción en esta línea:

```
0037108E    80F9 26          CMP CL,26     ;debe ser CL==26h
```

Necesitamos saber a qué hace referencia Cl en este momento, la respuesta está por encima justo en esta línea:

```
00371089    8A4E 05          MOV CL,BYTE PTR DS:[ESI+5]
```

nos situamos sobre ella y hacemos "Follow in Dump->Memory Address" a ver de donde recoge su valor CL:

```
Address   Hex dump                                         ASCII
0038A865  32 2D 31 32 31 32 31 32 2D 39 32 31 32 31 32 00  2-121212-921212.
0038A875  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0038A885  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0038A895  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0038A8A5  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lee justamente el 6° dígito, el valor 26 corresponde al dígito "&" así que cambiamos el "2" que hay allí por este, el serial quedará de esta manera:

72121&-121212-921212

Como termina el CALL_verificador ya que llegamos al RET suponemos que no habrá nunguna restricción más en cuanto al serial se refiere, de todos modos vamos a tracear un poco viendo los valores que maneja y demás, las dos primeras restricciones las pasamos ya de antes nos queda saber si en la tercera habrá alguna sorpresa de última hora o algo así, para evitar sustos traceamos el código viendo los valores que mueve y demás así nos aseguramos de hacer el trabajo bien hecho.

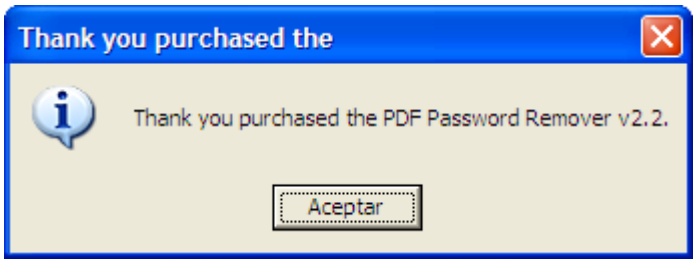
Como digo superamos las dos primeras restricciones sin problemas, llegamos a la tercera y presumiblemente la última de ellas:

```
00371089   8A4E 05     MOV CL,BYTE PTR DS:[ESI+5]
0037108C   33C0        XOR EAX,EAX
0037108E   80F9 26     CMP CL,26                    debe ser CL==26h
00371091   5F          POP EDI
00371092   5E          POP ESI
00371093   5B          POP EBX
00371094   0F94C0      SETE AL                      tiene que quedar AL==1..!!
00371097   83C4 18     ADD ESP,18
0037109A   C3          RETN
```

Llegamos con AL==26h fenomenal, llegamos al "SETE AL" que lo que hace es poner AL==1 en caso que esté activo el FLAG_ZERO, como la comapración enter CL y 26 resultó ser cierta activó este flag que ahora verifica dejando entonces AL==1 como se puede observar:

```
Registers (MMX)
EAX 00000001
ECX 0012EC26
EDX 0012EC32
EBX 00000000
ESP 0012EC0C
EBP 0012ED38
ESI 000E0698
EDI 0038A928 verypdf.0038A928
```

Ahí está AL==1 como debe de ser, jejeje.. este es el flag de éxito y lo tenemos activado, con este valor retornamos de la función y le indica al programa Serial_válido, superados todos los obstáculos damos RUN y...



nos da las gracias ;-DDD trabajo resuelto amigos...

También he de decir que hay inifinidad de seriales válidos, es una rutina un tanto venévola y nos deja bastante margen de movimiento para crear un KeyGen fácilmente, ejemplos de seriales válidos serían por ejemplo:

72124&457-457983
72121&-121212-921212
... etc ...

Y así una lista interminable..!!

Y hasta aquí llegamos en el día de hoy, espero que os haya gustado el tute tanto como a ami escribirlo y compartir con vosotros una experiencia de cracking, como siempre agradecer su apoyo a toda la lista de CracksLatinos sin excepción alguna, todos son grandes personas y en especial para mis amigos k0rt, Ricardo, [ZekkT], +NCR, [Juan Jose], Dan, kaos_xlro, bet0x y también para Morales, Shadow&Dark, Solid y como digo TODOS los demás, un saludo a todos y hasta la próxima..!!

Sugerencias, dudas, consultas o lo que sea me podéis escibir a mi dirección de email:

syxe00@yahoo.es